

UNIVERSIDAD EAFIT

Análisis de Algoritmos — 2026/01

PRÁCTICA FINAL INTEGRADORA (15%)

*Optimización de rutas y planificación de recursos
en redes de telecomunicaciones*

— *Algoritmos Codiciosos, Divide y Vencerás y Programación Dinámica* —

Integrantes:

Juan Jose Diaz

Samuel Calderón

1. Introducción

Un operador de fibra óptica debe resolver simultáneamente tres subproblemas de planificación de infraestructura: ordenar y consultar eficientemente su inventario de solicitudes de servicio, tender el menor cableado posible para interconectar nodos de red, y asignar ancho de banda limitado entre solicitudes competidoras maximizando el ingreso.

Este proyecto resuelve esos tres subproblemas sobre el dataset Telco Customer Churn (Kaggle, CC0), que contiene registros de 7,043 clientes reinterpretados como solicitudes de servicio. Cada paradigma se aplica donde sus propiedades lo hacen adecuado:

- Divide y Vencerás (Módulo A): MergeSort y búsqueda binaria garantizan $O(n \log n)$ en el peor caso y son ideales para ordenar y consultar grandes volúmenes de datos.
- Algoritmo Codicioso (Módulo B): Kruskal con Union-Find construye el árbol de expansión mínima eligiendo localmente la arista más barata; el Lema del ciclo garantiza el óptimo global.
- Programación Dinámica (Módulo C): La Mochila 0-1 requiere explorar subestructura óptima; el enfoque codicioso falla porque la decisión local no garantiza el óptimo global.

2. Descripción del dataset

Fuente: Kaggle — Telco Customer Churn, 7,043 registros, 21 atributos, licencia CC0.

Campos extraídos del CSV original (21 columnas):

Campo CSV (columna)	Interpretación	Notas
customerID (0)	Identificador de solicitud	Cadena alfanumérica
tenure (5)	Prioridad (meses de antigüedad)	Rango 0–72
MonthlyCharges (18)	Valor mensual (USD)	Promedio global: 64.76 USD
TotalCharges (19)	Valor acumulado (peso en mochila)	11 registros con campo vacío
Churn (20)	Estado: activo (No) / en riesgo (Yes)	5,174 activos (Churn=No)

Manejo de valores nulos: los 11 clientes con tenure = 0 presentan TotalCharges como espacio en blanco. Se les asigna TotalCharges = 0.0. El parser detecta el campo vacío comparando con `find_first_not_of(' ')`.

Construcción del grafo (Módulo B): los 7,043 registros se agrupan en 20 nodos asignando el registro i (en orden original del CSV) al grupo $i \bmod 20$. El costo de la arista (u, v) es $c(u, v) = \text{floor}(\bar{M}_u + \bar{M}_v)$, donde $\bar{M}_k$ es el promedio de MonthlyCharges del grupo k . Esto produce un grafo completo de 20 nodos y 190 aristas, determinista y reproducible.

3. Módulo A — Divide y Vencerás

3.1 MergeSort

Pseudocódigo:

```
MergeSort(A, l, r):
    si r - l <= 1: retornar
    m = (l + r) / 2
    MergeSort(A, l, m)
    MergeSort(A, m, r)
    Merge(A, l, m, r)    // fusión estable, mayor tenure primero

Merge(A, l, m, r):
    i = l, j = m, k = l
    mientras i < m o j < r:
        si j >= r o (i < m y A[i].tenure >= A[j].tenure):
            tmp[k++] = A[i++]
        sino: tmp[k++] = A[j++]
    copiar tmp[l..r] → A[l..r]
```

Análisis de complejidad: la recurrencia es $T(n) = 2T(n/2) + \Theta(n)$. Por el Teorema Maestro (caso 2: $a=2$, $b=2$, $f(n)=\Theta(n)=\Theta(n^{\log_2 2})$), la solución es $T(n) = \Theta(n \log n)$. MergeSort es preferible a QuickSort porque: (1) es estable — mantiene el orden relativo de registros con igual tenure —, y (2) garantiza $O(n \log n)$ en el peor caso, sin degradarse a $O(n^2)$ con datos casi ordenados o con muchos duplicados, situación posible aquí ya que muchos clientes comparten el mismo tenure.

Resultados empíricos de tiempo de ejecución:

n	Tiempo (ms)	$n \cdot \log_2(n)$	Ratio tiempo / Ratio teórico
1,000	0.74	9,966	—
3,500	2.16	41,195	Actual: 2.92× Teórico: 4.14×
7,043	4.10	90,009	Actual: 1.90× Teórico: 2.18×

Los ratios son consistentes con $O(n \log n)$. Las desviaciones se explican por el caché del procesador y la sobrecarga del sistema operativo en mediciones de milisegundos.

3.2 Búsqueda binaria recursiva

Pseudocódigo (sobre arreglo DESCENDENTE; retorna el último índice con tenure $\geq k$):

```
BusquedaBinaria(A, l, r, k):
    si l >= r: retornar -1
    m = (l + r) / 2
    si A[m].tenure >= k:
        der = BusquedaBinaria(A, m+1, r, k)
        retornar (der != -1) ? der : m
    sino:
        retornar BusquedaBinaria(A, l, m, k)
```

La función retorna el último índice cuyo tenure $\geq k$ — la frontera del segmento de alta prioridad.

Complejidad: $O(\log n)$.

Resultados de las 5 consultas:

Consulta	k	customerID	tenure
Q_A01	72	2234-XADUH	72
Q_A02	60	1036-GUDCL	60
Q_A03	45	5906-CVLHP	45
Q_A04	30	3976-BWUCK	30
Q_A05	12	7750-EYXWZ	12

4. Módulo B — Algoritmo Codicioso (Kruskal)

4.1 Descripción del grafo

- Nodos: 20
- Aristas: 190 (grafo completo K_{20})
- Costo promedio de arista: ≈ 128.84 unidades

4.2 Kruskal con Union-Find

Pseudocódigo:

```
Kruskal(G):
    ordenar aristas por costo ascendente
    DSU = UnionFind(n)      // unión por rango + compresión de caminos
    MST = []
    para cada arista (u, v, w) en orden:
        si DSU.find(u) != DSU.find(v):
            MST.agregar((u, v, w))
            DSU.unir(u, v)
            si |MST| == n-1: romper
    retornar MST

find(x):
    // compresión de caminos
    si padre[x] != x: padre[x] = find(padre[x])
    retornar padre[x]

unir(a, b):
    // unión por rango
    ra = find(a); rb = find(b)
    si rango[ra] < rango[rb]: padre[ra] = rb
    sino si rango[rb] < rango[ra]: padre[rb] = ra
    sino: padre[rb] = ra; rango[ra]++
```

Complejidad: $O(E \log E)$ para el ordenamiento; operaciones Union-Find en $O(\alpha(n))$ amortizado.

4.3 Resultado del MST

Peso total del MST: 2,388. Aristas incluidas ($19 = n-1$):

u	v	costo
1	12	122
12	16	123
0	1	124
1	9	124
1	14	124
1	19	125

12	13	125
1	11	125
1	7	125
1	3	126
1	18	126
1	17	126
1	10	126
4	12	127
2	12	127
1	8	127
1	5	127
6	12	129
1	15	130

4.4 Demostración del Lema del ciclo

Lema del ciclo (Cut property): para cualquier corte $(S, V \setminus S)$ del grafo, la arista de menor peso que cruza el corte pertenece a algún MST.

Aplicación concreta: considérese la arista $(1, 12)$ con costo 122. Si la removemos del MST, obtenemos dos componentes: $\{1, 0, 3, 5, 7, 8, 9, 10, 11, 14, 15, 17, 18, 19\}$ y $\{12, 2, 4, 6, 13, 16\}$. Cualquier otra arista que cruce este corte tiene costo ≥ 122 (la siguiente es $(12, 16) = 123$). Por tanto, $(1, 12)$ es la arista de peso mínimo del corte y el Lema garantiza que debe pertenecer al MST. Esto justifica la elección codiciosa de Kruskal.

4.5 Verificación en el subgrafo (nodos 0–4)

MST del subgrafo inducido por $\{0, 1, 2, 3, 4\}$:

u	v	costo
0	1	124
1	3	126
1	2	127
1	4	127

Peso total subgrafo: 504. Este resultado coincide con las aristas del MST completo restringidas al subconjunto $\{0, 1, 2, 3, 4\}$, verificando la correctitud de Kruskal.

5. Módulo C — Programación Dinámica

5.1 Formulación de la Mochila 0-1

Conjunto: 50 solicitudes activas (Churn = No) con mayor tenure del arreglo ordenado. Parámetros: peso $w_i = \text{round}(\text{TotalCharges}_i)$, valor $v_i = \text{round}(\text{MonthlyCharges}_i \times 10)$ en centavos, capacidad $W = 5,000$ unidades.

Nota: el enunciado indica $W = 500$, pero los 50 clientes seleccionados tienen $\text{tenure} = 72$ y TotalCharges entre 1,363 y 7,854 — ningún ítem cabe en 500 unidades. Se usa $W = 5,000$ para obtener una demostración algorítmica no trivial.

Recurrencia:

```
dp[0][w] = 0    para todo w
dp[i][w] = dp[i-1][w]
dp[i][w] = max(dp[i-1][w], dp[i-1][w - w_i] + v_i)    si w_i > w
                                                    si w_i ≤ w
```

5.2 Pseudocódigo — Tabulación y Backtracking

```
Tabulacion(pesos[], valores[], n, W):
    dp[0..n][0..W] = 0
    para i de 1 a n:
        para w de 0 a W:
            dp[i][w] = dp[i-1][w]
            si pesos[i-1] ≤ w:
                dp[i][w] = max(dp[i][w], dp[i-1][w-pesos[i-1]] + valores[i-1])
    retornar dp[n][W]

Backtracking(dp, pesos, n, W):
    seleccionados = []; w = W
    para i de n a 1:
        si dp[i][w] != dp[i-1][w]:
            seleccionados.agregar(i-1)
            w -= pesos[i-1]
    retornar seleccionados
```

5.3 Solución óptima

Valor óptimo total: 707 centavos | Ancho de banda usado: 4,851 / 5,000

customerID	w (TotalCharges)	v (MonthlyCharges×10)
9286-BHDQG	3,140	453
4312-KFRXN	1,711	254

5.4 Contraejemplo: fallo del enfoque codicioso

Tres solicitudes del conjunto con capacidad = 14,223 unidades:

customerID	w	v	ratio v/w
5248-YGIJN	6,369	903	0.1418
6234-RAAPL	7,252	999	0.1378
5954-BDFSG	7,854	1,075	0.1369

Greedy ordena por ratio y elige 5248-YGIJN + 6234-RAAPL (ambos caben). No puede agregar 5954-BDFSG. La PD descubre que 5248-YGIJN + 5954-BDFSG da mayor valor total:

Enfoque	Solicitudes seleccionadas	Valor total	¿Óptimo?
Codicioso (ratio v/w)	5248-YGIJN + 6234-RAAPL	1,902	No
PD (Mochila 0-1)	5248-YGIJN + 5954-BDFSG	1,978	Sí

El greedy pierde 76 centavos porque asigna demasiado peso a 6234-RAAPL (ratio=0.1378) en lugar de 5954-BDFSG (ratio=0.1369 pero con mayor valor absoluto al combinarse). La subestructura óptima de la mochila requiere considerar el efecto global de cada elección.

5.5 Análisis de complejidad y pseudopolinomialidad

Tiempo: $\Theta(n \times W) = \Theta(50 \times 5,000) = \Theta(250,000)$ operaciones.

Espacio: $\Theta(n \times W)$ para la tabla `dp[51][5,001]`.

Pseudopolinomialidad: la complejidad $\Theta(n \times W)$ depende del valor numérico de W , no de su longitud en bits ($\log W$). Si W creciera exponencialmente en la longitud de la entrada, el algoritmo dejaría de ser polinomial. Por eso la Mochila 0-1 se clasifica como pseudopolinomial: eficiente en la práctica cuando W es razonablemente pequeño, pero no polinomial en el sentido estricto de la teoría de complejidad.

6. Integración del pipeline

El programa se ejecuta en un solo comando:

```
g++ -std=c++17 -O2 -o ada_pf src/main.cpp src/*.cpp
./ada_pf data/WA_Fn-UseC_-Telco-Customer-Churn.csv
```

Diagrama de flujo del pipeline completo:

```
CSV (7,043 registros)
|
+---> Módulo B: compute_group_monthly_avg (orden original del CSV)
|               --> Kruskal MST --> mst_red.txt
|
+---> MergeSort descendente por tenure
|               |
|               +---> solicitudes_ordenadas.csv
|               +---> Búsqueda binaria --> busquedas_A.txt
|               +---> timing_mergesort.txt
|               +---> Top-50 (Churn=No) --> Módulo C: Mochila 0-1
|                                   --> asignacion_bw.txt
```

Dependencia clave: el Módulo B usa el orden original del CSV para asignar registros a grupos por índice $i \bmod 20$ — antes del sort. El Módulo C usa la salida del sort del Módulo A para seleccionar las 50 solicitudes activas de mayor tenure.

7. Conclusiones

Paradigma	Garantía de optimalidad	Complejidad	Observación
D&V (MergeSort)	Siempre óptimo ($O(n \log n)$ peor caso)	$\Theta(n \log n)$	Tiempos empíricos consistentes
Codicioso (Kruskal)	Óptimo global para MST por Lema del ciclo	$O(E \log E)$	Falla en Mochila 0-1 (contraejemplo demostrado)
PD (Mochila 0-1)	Óptimo global garantizado	$\Theta(n \cdot W)$ pseudopol.	Con datos reales W debe escalarse

La diferencia clave entre Kruskal y la Mochila está en la propiedad de subestructura: en MST la elección local (arista mínima sin ciclo) conduce al óptimo global porque los subproblemas son independientes. En la Mochila 0-1 los subproblemas se superponen y la elección local puede bloquear combinaciones globalmente mejores, haciendo necesaria la PD.

8. Herramientas de IA utilizadas

Se utilizó Claude (Anthropic) como herramienta de asistencia durante el desarrollo, en los siguientes contextos permitidos:

- Consulta de conceptos teóricos sobre MergeSort, Kruskal y Mochila 0-1.
- Depuración de errores de compilación (índices incorrectos de columnas del CSV, dirección de búsqueda binaria en arreglo descendente).
- Retroalimentación sobre la redacción del informe.

Toda la comprensión, diseño de la solución e integración del pipeline fue realizada por los integrantes del equipo.

9. Referencias

1. Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2022). Introduction to Algorithms (4th ed.). MIT Press. Caps. 2, 4, 16, 23.
2. Blastchar. (2018). Telco Customer Churn [Dataset]. Kaggle.
<https://www.kaggle.com/datasets/blastchar/telco-customer-churn>
3. Material del curso Análisis de Algoritmos 2026/01, Universidad EAFIT.

10. Roles del equipo

Integrante	Módulos y contribuciones
Juan Jose Díaz	Módulo A (MergeSort + búsqueda binaria), integración del pipeline, README
Samuel Calderón	Módulo B (Kruskal + Union-Find), Módulo C (Mochila 0-1 + contraejemplo), informe técnico